Lecture 09: finite state machines

Goals

- differentiate between *stateless* and *stateful* computation
- enumerate the characteristics of a finite state machine (FSM)
- "execute" an FSM on a particular input
- define *sequential logic*
- describe how combinational logic elements and storage elements can be combined to implement sec

now we've got combinational logic and storage elements, we can start building bigger and better things

like… combination locks!

Google shows us what combination locks are

even in the first page of results, there is some variation

some care about the order in which you enter the numbers, some do not

the latter will be more complicated because we have to remember what was entered previously

let's tackle the easy one first, though

how would you build a circuit to do this?

you need 4 inputs

each with an AND-gate to make sure the digit is correct

then another AND-gate to make sure all digits are correct

(yes, you could do this with a single 16-input AND-gate, but this is much clearer)

then the circuit outputs 1 if we've got the correct combination, 0 otherwise

stateless_combo_lock.circ

(the combo is 1-9-7-7)

let's think about the more complicated version now

it has 2 inputs: a direction and a number

we can't decide whether the correct combination has been entered just by examining the inputs at a part

we need to remember "where we are" in the sequence of steps required to unlock it

we can precisely enumerate a bunch of characteristics of this lock:

- all possible inputs
- all possible outputs
- all places we *could* be in unlock process
- all transitions between these places

we can draw a picture of this

circles represent *states*: places we could be in the unlock process

arrowed lines between circles represent *transitions*: what makes us "go" from one state to another

the only possible places we could be in the unlock process are:

- we have entered 0 correct numbers
- we have entered the first number correctly
- we have entered the first two numbers correctly
- we have entered all three numbers correctly

those of you who've taken CS 200 will recognize this as a deterministic finite automata (DFA)

also sometimes called a Finite State Machine (FSM)

because there are…

- a finite set of states
- a finite set of inputs
- a finite set of outputs
- a finite set of state transitions

can we build a circuit to behave like this ~~combination lock~~ state machine? (hint: yes)

let's first think about what data we need to store

we don't need to remember any of the inputs, we just need to remember what state we're in: we can use

how big? there are four states, so we need to store four distinct values, so we need a 2-bit register (beca

then the decision of what state to go to next is a combination of the current state and the input

combination of

combinational logic

stateful_combo_lock.circ

things to notice

the input to the "state" register depends upon both the input and the current state

the "is-unlocked" output depends upon both the current state and some combinational logic

this is a pattern we will see over and over and over again

in fact, this is the *only* pattern we will see from here on out (though it'll get much more complicated)

think back to programs you've written

they take in new stuff, combine it with old stuff, and save the result

Last modified: 10/02/2026 13:37:06